

Policies API — Frontend Developer Guide

Overview

The Policies module manages company-wide employment policies (Terms & Conditions, Privacy Policy, NDA, etc.). Each policy has a **fixed** `policyType` **enum** so you can fetch any policy by its known type without needing a database ID.

Policy Types (Enum)

Every policy has a unique `policyType` value. Use these as stable identifiers:

policyType value	Human Label	Typical Usage
terms-and-conditions	Terms & Conditions	Employment terms employees must accept
privacy-policy	Privacy Policy	Data handling and privacy terms
code-of-conduct	Code of Conduct	Behavioral expectations
nda	Non-Disclosure Agreement	Confidentiality agreement
it-security	IT Security Policy	IT security rules and guidelines
acceptable-use	Acceptable Use Policy	Acceptable usage of company resources
data-protection	Data Protection Policy	GDPR / data protection compliance
other	Other	Catch-all for custom policies

Imports

```
import {
  POLICY_TYPES,           // readonly array of all type strings
  POLICY_TYPE_LABELS,    // Record<PolicyType, string> – human-readable labels
  type PolicyType,       // TypeScript union type
  getGuestPolicyByType,  // fetch active policy by type – no auth needed
} from "@lib/api/policies.api";
```

API Endpoints

Guest (requires `x-api-key` header, no user login)

Method	Endpoint	Description
GET	/policies/guest/:policyType	Get an active policy by type — requires <code>x-api-key</code>

Public (Employee-facing, requires authentication)

Method	Endpoint	Description
GET	/policies/public	List all active policies + acceptance status
POST	/policies/public/:id/accept	Accept a specific policy

Admin (requires policies:manage permission)

Method	Endpoint	Description
GET	/policies	List all policies (paginated)
GET	/policies/type/:policyType	Get policy by its fixed enum type
GET	/policies/:id	Get policy by MongoDB ID
POST	/policies	Create a new policy
PUT	/policies/:id	Update a policy
DELETE	/policies/:id	Archive (soft-delete) a policy
GET	/policies/compliance	Compliance report
GET	/policies/:id/acceptances	List who accepted a policy
GET	/policies/:id/versions	Version history
GET	/policies/:id/versions/:versionId	Get a specific version snapshot
POST	/policies/:id/versions/:versionId/restore	Restore a version

Response Structures

All responses follow this wrapper format:

```
{
  "success": true,
  "message": "...",
  "data": { /* actual payload */ }
}
```

The frontend `unwrap` helper extracts the `data` field automatically. All examples below show the **unwrapped payload**.

GET /policies/guest/:policyType — Guest Policy (API Key)

Returns a single active policy by type. **No user login required**, but the request must include a valid `x-api-key` header.

Required Header:

```
x-api-key: <your-guest-api-key>
```

Only returns active policies with public-safe fields (no `createdBy`, `updatedBy`, or `acceptanceCount`).

Example: GET /policies/guest/terms-and-conditions

```
{
  "_id": "6840a1b2c3d4e5f678901234",
  "policyType": "terms-and-conditions",
  "title": "Terms and Conditions",
  "slug": "terms-and-conditions",
  "version": 2,
  "isRequired": true,
  "publishedAt": "2026-06-01T10:00:00.000Z",
  "contentHtml": "<h1>Terms and Conditions</h1><p>...</p>",
  "contentJson": { "type": "doc", "content": [...] },
  "contentText": "Terms and Conditions ..."
}
```

Error Responses:

- 401 — Missing x-api-key header
- 403 — Invalid API key
- 404 — Policy type not found or not in active status
- 503 — GUEST_API_KEY not configured on the server

GET /policies/public — Active Policies (Employee)

Returns all active policies with the current user's acceptance status.

```
[
  {
    "_id": "6840a1b2c3d4e5f678901234",
    "policyType": "terms-and-conditions",
    "title": "Terms and Conditions",
    "slug": "terms-and-conditions",
    "version": 2,
    "isRequired": true,
    "publishedAt": "2026-06-01T10:00:00.000Z",
    "contentHtml": "<h1>Terms and Conditions</h1><p>...</p>",
    "contentJson": { "type": "doc", "content": [...] },
    "contentText": "Terms and Conditions ...",
    "isAccepted": false,
    "acceptedAt": null
  },
  {
    "_id": "6840a1b2c3d4e5f678905678",
    "policyType": "privacy-policy",
    "title": "Privacy Policy",
    "slug": "privacy-policy",
    "version": 1,
    "isRequired": true,
    "publishedAt": "2026-06-01T10:00:00.000Z",
    "contentHtml": "<h1>Privacy Policy</h1><p>...</p>",
    "contentJson": { "type": "doc", "content": [...] },
    "contentText": "Privacy Policy ...",
    "isAccepted": true,
    "acceptedAt": "2026-06-02T14:30:00.000Z"
  }
]
```

GET /policies/type/:policyType — Policy by Type

Returns a single policy detail by its enum type. Example: GET /policies/type/terms-and-conditions

```
{
  "_id": "6840a1b2c3d4e5f678901234",
  "policyType": "terms-and-conditions",
  "title": "Terms and Conditions",
  "slug": "terms-and-conditions",
  "version": 2,
  "status": "active",
  "isRequired": true,
  "publishedAt": "2026-06-01T10:00:00.000Z",
  "contentHtml": "<h1>Terms and Conditions</h1><p>...</p>",
  "contentJson": { "type": "doc", "content": [...] },
  "contentText": "Terms and Conditions ...",
  "createdBy": { "_id": "...", "name": "Admin User", "email": "admin@example.com" },
  "updatedBy": { "_id": "...", "name": "Admin User", "email": "admin@example.com" },
  "createdAt": "2026-05-15T08:00:00.000Z",
  "updatedAt": "2026-06-01T10:00:00.000Z",
  "acceptanceCount": 42
}
```

GET /policies — List All (Admin)

Query params: ?page=1&limit=20&status=active

```
{
  "policies": [
    {
      "_id": "...",
      "policyType": "terms-and-conditions",
      "title": "Terms and Conditions",
      "slug": "terms-and-conditions",
      "version": 2,
      "status": "active",
      "isRequired": true,
      "publishedAt": "2026-06-01T10:00:00.000Z",
      "createdBy": { "_id": "...", "name": "Admin", "email": "admin@co.com" },
      "updatedBy": { "_id": "...", "name": "Admin", "email": "admin@co.com" },
      "createdAt": "2026-05-15T08:00:00.000Z",
      "updatedAt": "2026-06-01T10:00:00.000Z",
      "acceptanceCount": 42
    }
  ],
  "pagination": {
    "page": 1,
    "limit": 20,
    "total": 3,
    "totalPages": 1
  }
}
```

GET /policies/compliance — Compliance Report (Admin)

```
{
  "policies": [
    {
      "_id": "...",
      "title": "Terms and Conditions",
      "version": 2,
      "acceptedCount": 42,
      "pendingCount": 8
    }
  ],
  "totalEmployees": 50,
  "fullyCompliant": 38,
  "nonCompliant": [
    {
      "_id": "...",
      "name": "John Doe",
      "email": "john@example.com",
      "position": "Developer",
      "department": { "name": "Engineering" }
    }
  ]
}
```

POST /policies/public/:id/accept — Accept Policy

```
{
  "_id": "...",
  "policy": "6840a1b2c3d4e5f678901234",
  "user": "...",
  "policyVersion": 2,
  "acceptedAt": "2026-06-04T12:00:00.000Z",
  "ipAddress": "192.168.1.1"
}
```

Frontend Usage Examples

1. Fetch a policy without login (guest, with API key)

```
import { getGuestPolicyByType } from "@lib/api/policies.api";

// No user login needed - x-api-key is injected automatically from env
const terms = await getGuestPolicyByType("terms-and-conditions");
console.log(terms.title);           // "Terms and Conditions"
console.log(terms.contentHtml);    // "<h1>Terms and Conditions</h1>..."
```

Setup: Set `NEXT_PUBLIC_GUEST_API_KEY` in your `.env.local` to match the backend's `GUEST_API_KEY`.

2. Fetch a specific policy by type (admin, authenticated)

```
import { getPolicyByType } from "@lib/api/policies.api";

// Requires policies:manage permission
const terms = await getPolicyByType("terms-and-conditions");
console.log(terms.title);      // "Terms and Conditions"
console.log(terms.contentHtml); // "<h1>Terms and Conditions</h1>..."
console.log(terms.version);    // 2
```

3. React Query hook for guest policy (no auth)

```
import { useQuery } from "@tanstack/react-query";
import { getGuestPolicyByType } from "@lib/api/policies.api";

function PublicTermsPage() {
  const { data: policy, isLoading } = useQuery({
    queryKey: ["policies", "guest", "terms-and-conditions"],
    queryFn: () => getGuestPolicyByType("terms-and-conditions"),
  });

  if (isLoading) return <p>Loading...</p>;

  return (
    <div>
      <h1>{policy.title}</h1>
      <div dangerouslySetInnerHTML={{ __html: policy.contentHtml }} />
    </div>
  );
}
```

4. React Query hook for admin policy type

```
import { useQuery } from "@tanstack/react-query";
import { getPolicyByType, POLICY_TYPE_LABELS } from "@lib/api/policies.api";

function TermsPage() {
  const { data: policy, isLoading } = useQuery({
    queryKey: ["policies", "type", "terms-and-conditions"],
    queryFn: () => getPolicyByType("terms-and-conditions"),
  });

  if (isLoading) return <p>Loading...</p>;

  return (
    <div>
      <h1>{policy.title}</h1>
      <div dangerouslySetInnerHTML={{ __html: policy.contentHtml }} />
    </div>
  );
}
```

5. List all active policies with acceptance status

```
import { useActivePolicies } from "@hooks/queries";

function MyPolicies() {
  const { data: policies, isLoading } = useActivePolicies();

  if (isLoading) return <p>Loading...</p>;

  return (
    <ul>
      {policies?.map((p) => (
        <li key={p._id}>
          {p.title} – v{p.version}
          {p.isAccepted ? "  Accepted" : "  Pending"}
        </li>
      ))}
    </ul>
  );
}
```

6. Accept a policy

```
import { usePolicyMutations } from "@hooks/queries";

function AcceptButton({ policyId }: { policyId: string }) {
  const { acceptPolicy } = usePolicyMutations();

  return (
    <button
      onClick={() => acceptPolicy.mutate(policyId)}
      disabled={acceptPolicy.isPending}
    >
      {acceptPolicy.isPending ? "Accepting..." : "I Accept"}
    </button>
  );
}
```

7. Display human-readable type label

```
import { POLICY_TYPE_LABELS, type PolicyType } from "@lib/api/policies.api";

function PolicyBadge({ type }: { type: PolicyType }) {
  return <span>{POLICY_TYPE_LABELS[type]}</span>;
  // Renders: "Terms & Conditions", "Privacy Policy", etc.
}
```

Available React Query Hooks

All hooks are exported from `@hooks/queries` :

Hook	Purpose	Returns
useActivePolicies()	All active policies + acceptance status	ActivePolicy[]
usePolicies(filters)	Admin: paginated policy list	{ policies, pagination }
usePolicyDetail(id)	Single policy by ID	PolicyDetail
usePolicyAcceptances(id)	Who accepted a policy	{ acceptances, pagination }
useComplianceReport()	Overall compliance stats	ComplianceReport
usePolicyVersions(id)	Version history for a policy	PolicyVersion[]
usePolicyVersion(policyId, versionId)	Single version detail	PolicyVersionDetail
usePolicyMutations()	All write operations	{ createPolicy, updatePolicy, deletePolicy, acceptPolicy, restoreVersion }

TypeScript Interfaces

```
type PolicyType =
  | "terms-and-conditions"
  | "privacy-policy"
  | "code-of-conduct"
  | "nda"
  | "it-security"
  | "acceptable-use"
  | "data-protection"
  | "other";

interface PolicySummary {
  _id: string;
  policyType: PolicyType;
  title: string;
  slug: string;
  version: number;
  status: "draft" | "active" | "archived";
  isRequired: boolean;
  publishedAt: string | null;
  createdBy: UserRef | null;
  updatedBy: UserRef | null;
  createdAt: string;
  updatedAt: string;
  acceptanceCount: number;
}

interface PolicyDetail extends PolicySummary {
  contentHtml: string;
  contentJson: object;
  contentText: string;
}

/** Returned by the guest (unauthenticated) endpoint – no admin fields */
interface GuestPolicy {
  _id: string;
  policyType: PolicyType;
  title: string;
  slug: string;
  version: number;
  isRequired: boolean;
  publishedAt: string | null;
  contentHtml: string;
  contentJson: object;
  contentText: string;
}

interface ActivePolicy {
  _id: string;
  policyType: PolicyType;
  title: string;
  slug: string;
  version: number;
  isRequired: boolean;
  publishedAt: string | null;
  contentHtml: string;
  contentJson: object;
  contentText: string;
}
```

```
isAccepted: boolean;  
acceptedAt: string | null;  
}
```

Enforcement Flow

The `PolicyGuard` component (`@/components/PolicyGuard.tsx`) is mounted in both the admin and protected layouts. It:

1. Fetches active policies via `useActivePolicies()`
2. Checks if any **required** policies have `isAccepted: false`
3. If so, redirects to `/policy-acceptance?from=<current-path>`
4. After all policies are accepted, the user is redirected back

Note: This is a client-side guard. The backend does NOT block API calls for unaccepted policies — it only provides acceptance status. The enforcement is purely UI-level via the redirect.